

树（二） | 表达式 · 二叉搜索树 · 哈夫曼树

适用对象：CSP-J 普及组备考学生

课时安排：1 课时（约 60 min）

核心考点：前中后缀表达式 · 二叉搜索树（BST） · 哈夫曼树与哈夫曼编码

目录

- [Part 1：前中后缀表达式](#)
- [Part 2：二叉搜索树（BST）](#)
- [Part 3：哈夫曼树与哈夫曼编码](#)
- [🧠 课堂速记要点](#)
- [📋 速查答案记忆卡](#)
- [附录：课堂板书建议 & 讲解节奏](#)

Part 1：前中后缀表达式（≈25min）

1.1 三种表达式

类型	运算符位置	示例 a+b
中缀	运算符在两操作数中间	a + b
前缀（波兰式）	运算符在操作数前	+ a b

类型	运算符位置	示例 a+b
后缀 (逆波兰式)	运算符在操作数后	a b +

后缀表达式不需要括号，运算符优先级已隐含在顺序中，计算机求值最方便。

💡 **关联：** 表达式的树形结构——前序遍历对应前缀式、中序遍历对应中缀式、后序遍历对应后缀式。这也是树遍历的直接应用。

1.2 中缀转后缀、中缀转前缀

标优先级

中缀: $a + b * c + (d + e) / f$

1. 标优先级

$$a + b^4 * c^2 + (d^5 + e^1) / f^3$$

2. 按优先级操作

前缀

1. $+de$
2. $*bc$
3. $/ +def$
4. $+ a * bc$
5. $++ a * bc \ / + def$

后缀

1. $de +$
2. $bc *$
3. $de + f /$
4. $a bc * +$
5. $abc * + de + f / +$

1.3 后缀表达式求值 (栈)

步骤:

1. 从左到右扫描
2. 遇到操作数：入栈

3. 遇到**运算符 op**：弹出栈顶 b ，再弹出 a ，计算 $a \text{ op } b$ ，结果入栈

4. 扫描完，栈中唯一元素即为结果

⚠ 注意顺序：先弹出的是**右操作数 b** ，后弹出的是**左操作数 a** ，计算 $a \text{ op } b$ 。减法和除法顺序不能反！

具体例子：后缀表达式 $2 \ 3 \ + \ 8 \ 4 \ - \ * \ 5 \ /$ （对应中缀 $(2 + 3) \times (8 - 4) \div 5$ ）

扫描到	栈（底→顶）	操作
2	2	操作数 → 入栈
3	2 3	操作数 → 入栈
+	5	弹 $b=3$ 、弹 $a=2$ ，算 $2 + 3 = 5$ → 入栈
8	5 8	操作数 → 入栈
4	5 8 4	操作数 → 入栈
-	5 4	弹 $b=4$ 、弹 $a=8$ ，算 $8 - 4 = 4$ → 入栈
*	20	弹 $b=4$ 、弹 $a=5$ ，算 $5 \times 4 = 20$ → 入栈
5	20 5	操作数 → 入栈
/	4	弹 $b=5$ 、弹 $a=20$ ，算 $20 \div 5 = 4$ → 入栈

扫描完栈中只剩 4，结果 = 4 **✓**（验算： $(2+3) \times (8-4) \div 5 = 5 \times 4 \div 5 = 4$ ）

⚠ 减法/除法陷阱： $8 \ 4 \ -$ 算的是 $8 - 4 = 4$ ，**不是** $4 - 8 = -4$ ；
 $20 \ 5 \ /$ 算的是 $20 \div 5 = 4$ ，**不是** $5 \div 20 = 0.25$ ！先弹出的是**右操作数**，后弹出的是**左操作数**，顺序反了就错。

💡 **观察：** 中缀里的括号在后缀式中"消失"了—— $(2+3)$ 变成 $2\ 3\ +$ 、 $(8-4)$ 变成 $8\ 4\ -$ ，运算优先级已隐含在顺序中；且 $5 \times 4 = 20$ 的中间结果又作为 $/$ 的左操作数入栈，这正是栈"暂存中间结果"的意义。

1.4 前缀表达式求值

方法： 从右到左扫描，遇到操作数入栈，遇到运算符弹出两个操作数计算（先弹出的是左操作数）。

具体例子： 前缀表达式 $/\ * \ + \ 2\ 3 \ - \ 8\ 4\ 5$ （对应中缀 $(2 + 3) \times (8 - 4) \div 5$ ，即 1.3 后缀例子的前缀写法）

扫描到（从右到左）	栈（底→顶）	操作
5	5	操作数 → 入栈
4	5 4	操作数 → 入栈
8	5 4 8	操作数 → 入栈
-	5 4	先弹 a=8 （左操作数）、再弹 b=4，算 $8 - 4 = 4 \rightarrow$ 入栈
3	5 4 3	操作数 → 入栈
2	5 4 3 2	操作数 → 入栈
+	5 4 5	先弹 a=2 （左操作数）、再弹 b=3，算 $2 + 3 = 5 \rightarrow$ 入栈
*	5 20	先弹 a=5 （左操作数）、再弹 b=4，算 $5 \times 4 = 20 \rightarrow$ 入栈
/	4	先弹 a=20 （左操作数）、再弹 b=5，算 $20 \div 5 = 4 \rightarrow$ 入栈

扫描完栈中只剩 4，结果 = 4 ✓ (验算： $(2+3) \times (8-4) \div 5 = 4$ ，与 1.3 后缀例子结果一致)

⚠ 与前缀（后缀）的顺序对比：

- 后缀：从左到右扫，先弹出的是右操作数
- 前缀：从右到左扫，先弹出的是左操作数
- 减法/除法举例： $- \ 8 \ 4 \rightarrow$ 先弹 $a=8$ 、再弹 $b=4 \rightarrow 8 - 4 = 4$ (不是 $4 - 8$)； $/ \ 20 \ 5 \rightarrow$ 先弹 $a=20$ 、再弹 $b=5 \rightarrow 20 \div 5 = 4$ (不是 $5 \div 20$)

💡 观察：同一中缀式 $(2+3) \times (8-4) \div 5$ 的三种写法：中缀 $(2+3) \times (8-4) \div 5$ 、后缀 $2 \ 3 \ + \ 8 \ 4 \ - \ * \ 5 \ /$ 、前缀 $/ \ * \ + \ 2 \ 3 \ - \ 8 \ 4 \ 5$ 。后缀从左往右读、先弹右操作数；前缀从右往左读、先弹左操作数——方向相反、弹出顺序也相反，但算的是同一笔账。

1.5 小巧思

关于前缀后缀的表达式计算，记住一个样例： $a-b$

前缀： $-ab$

都知道，碰到操作符要弹出两个并计算，所以前缀需要从**后面开始**，那么就是栈底： $b \ a$ 。此时碰到了操作符 $-$ ，那么先弹出的是 a ，后弹出的是 b ，原中缀是 $a-b$ ，所以**先弹出的是左操作数，后弹出的是右操作数**。

后缀： $ab-$

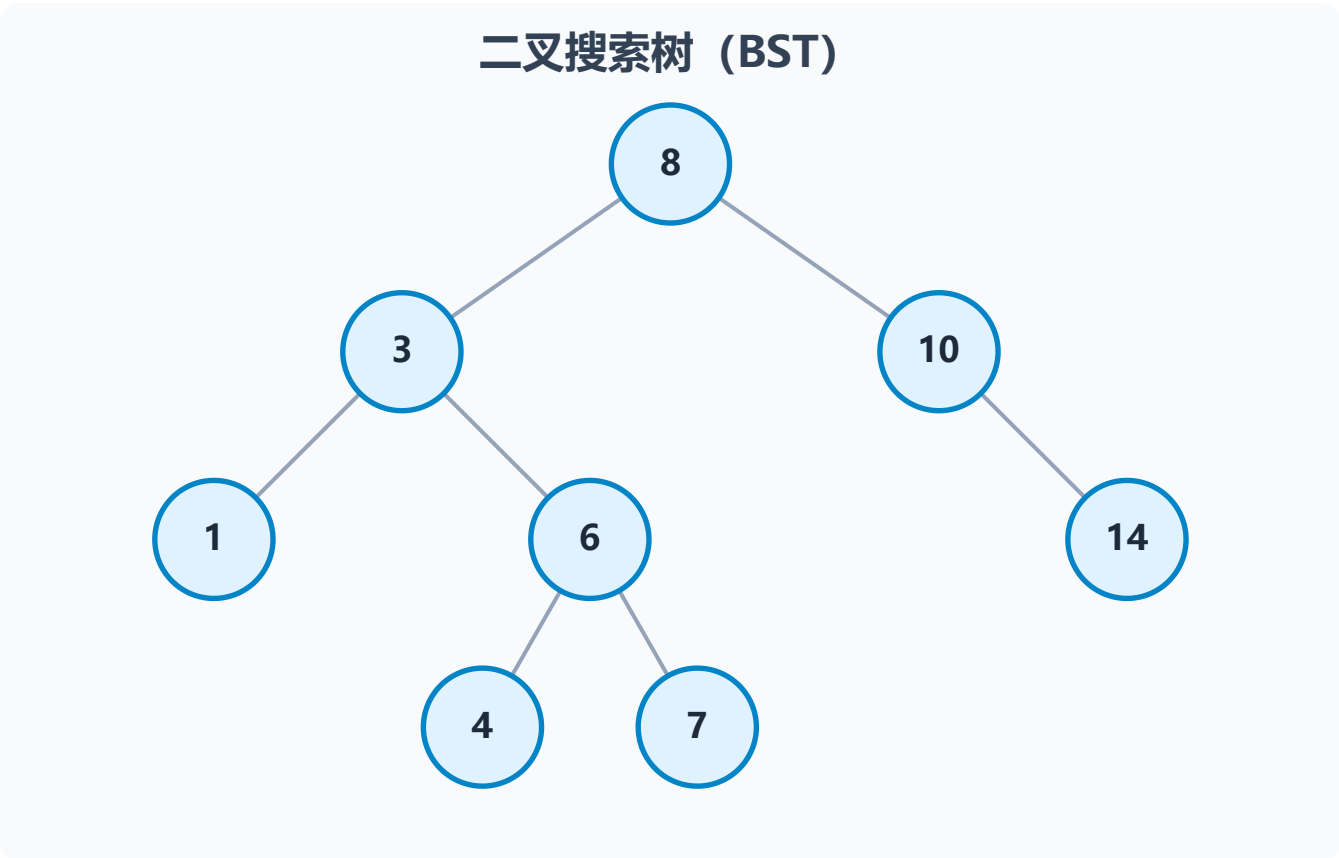
同理，为了碰到操作符-有操作数，后缀就需要从左边开始，那么就是栈底： $a \ b$ 。此时碰到了操作符 $-$ ，那么先弹出的是 b ，后弹出的是 a ，原中缀是 $a-b$ ，所以**先弹出的是右操作数，后弹出的是左操作数**。

Part 2: 二叉搜索树 (BST) (≈15min)

2.1 定义与性质

二叉搜索树 (Binary Search Tree, BST)，又称**二叉排序树**，是满足以下性质的二叉树：

BST 性质： 对树中**每一个节点**，其**左子树中所有节点的值 < 该节点的值 < 右子树中所有节点的值**，且左右子树本身也都是 BST（递归定义）。



上图是一棵 BST：根为 8，左子树（3、1、6、4、7）全部小于 8，右子树（10、14）全部大于 8；子树内部同样满足。

概念	说明
二叉搜索树	左 < 根 < 右，整棵树递归满足
二叉排序树	BST 的别称（因为中序遍历有序，见 2.2）
BST	Binary Search Tree 缩写

⚠ **易错点（判断 BST 的经典陷阱）**：不能只检查"每个节点比自己的左右孩子大/小"就完事，必须保证**整棵子树**都满足。例如：根为 8、左孩子为 3、3 的右孩子为 9 —— 局部看 $3 < 9$ 没问题，但 9 出现在根的左子树却大于 8，**违反 BST 性质**，这棵树不是 BST。

2.2 中序遍历有序（核心性质）

💡 **BST 最重要的一条性质**：对 BST 进行中序遍历，得到的是**严格递增序列**。

对示例树中序遍历：`1 → 3 → 4 → 6 → 7 → 8 → 10 → 14` ✅ 严格递增

反过来同样成立（这是"判断 BST"的理论依据）：

判定方法：一棵二叉树的中序遍历序列**严格递增** ⇔ **这棵树是 BST**。

应用：

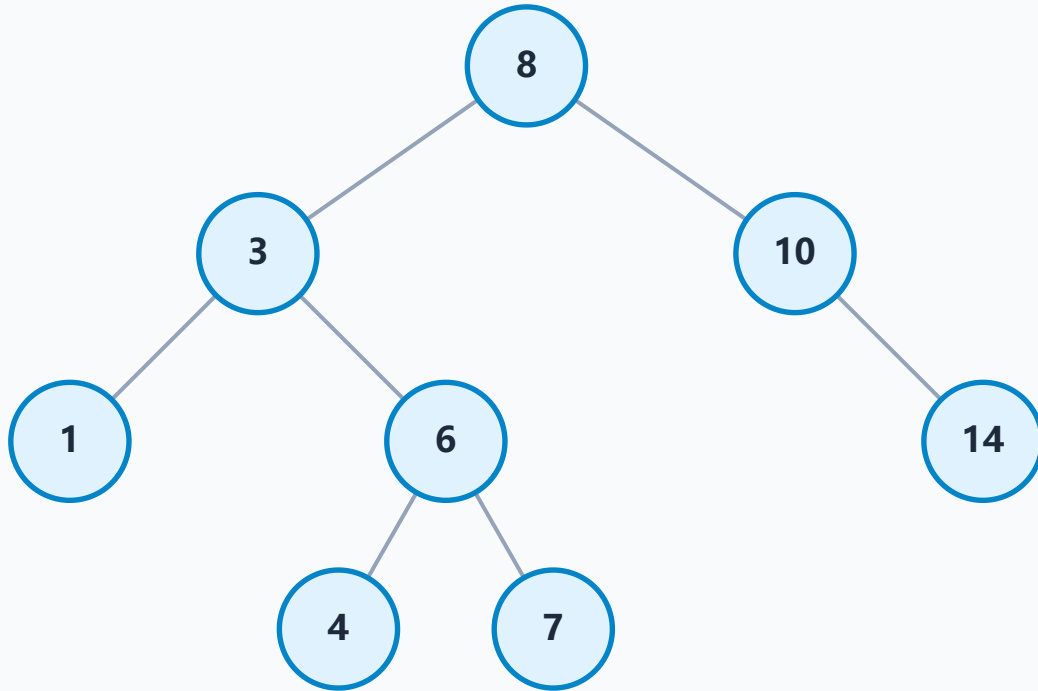
1. **判断 BST**：写出中序序列，看是否严格递增（选择题常考）
2. **求第 k 小元素**：BST 中序遍历的第 k 个节点就是第 k 小元素
3. **排序**：把 n 个数插入 BST，再中序遍历输出，就得到有序序列（Tree Sort 思想）

⚠ **注意"严格"二字**：中序序列出现相等元素（如 `1, 3, 3, 5`）时，按定义不满足严格递增，该树不算 BST（选择题里常拿"非严格"当陷阱）。

2.3 真题精练

例1（单选） 对下图所示二叉搜索树进行中序遍历，得到的序列是（ ）

二叉搜索树 (BST)



- A. 1,3,4,6,7,8,10,14
- B. 8,3,1,6,4,7,10,14
- C. 1,4,3,6,7,8,10,14
- D. 14,10,8,7,6,4,3,1

解析：中序遍历顺序为"左 → 根 → 右"。从根 8 出发，先遍历左子树：1 → 3 → 4 → 6 → 7，再访问根 8，最后右子树 10 → 14。得 1, 3, 4, 6, 7, 8, 10, 14。B 是前序遍历，D 是逆序，C 中 4 与 3 的顺序错（4 在 3 的右子树，应先 3 后 4）。选 **A**。

答案：A

例2（单选） 下列关于二叉搜索树 (BST) 的说法，正确的是（ ）

- A. 对 BST 进行后序遍历，得到的序列一定有序
- B. 一棵二叉树的中序遍历序列是递增的，则它一定是 BST
- C. BST 中，根节点一定是整棵树的最大值

- D. 中序遍历一棵 BST，序列中可能出现相等的相邻元素

解析： B 对——中序严格递增 $\Leftrightarrow$ BST，这是判定 BST 的依据（见 2.2）。A 错，有序的是**中序**而非后序；C 错，根不一定最大，最大元素在"最右下"；D 错，严格递增意味着不允许相等。选 **B**。

答案：B

 **本节速记：** BST = 左 < 根 < 右（递归成立，检查整棵子树）· 中序遍历 = 严格递增序列（最核心）· 判断 BST 看中序是否严格递增 · 第 k 小元素 = 中序第 k 个

Part 3：哈夫曼树与哈夫曼编码 (≈20min)

3.1 带权路径长度（WPL）与最优二叉树

路径长度： 从根到某节点的边数（根到根的路径长度为 0）。

带权路径长度（WPL）： 所有**叶子节点**的（权值 × 路径长度）之和：

$$WPL = \sum (\text{叶子权值} \times \text{叶子到根的路径长度})$$

哈夫曼树（霍夫曼树 / 最优二叉树）： 在叶子权值相同的前提下，**WPL 最小**的二叉树。

概念	含义
路径长度	根到节点的边数
带权路径长度 WPL	$\sum \text{叶子权值} \times \text{路径长度}$
哈夫曼树	WPL 最小的二叉树（最优二叉树）

💡 **直觉理解：** 权值大的叶子应该离根近（路径短），权值小的叶子可以离根远——这样 WPL 才小。

3.2 哈夫曼树的构造（贪心）

构造步骤（核心必会）：

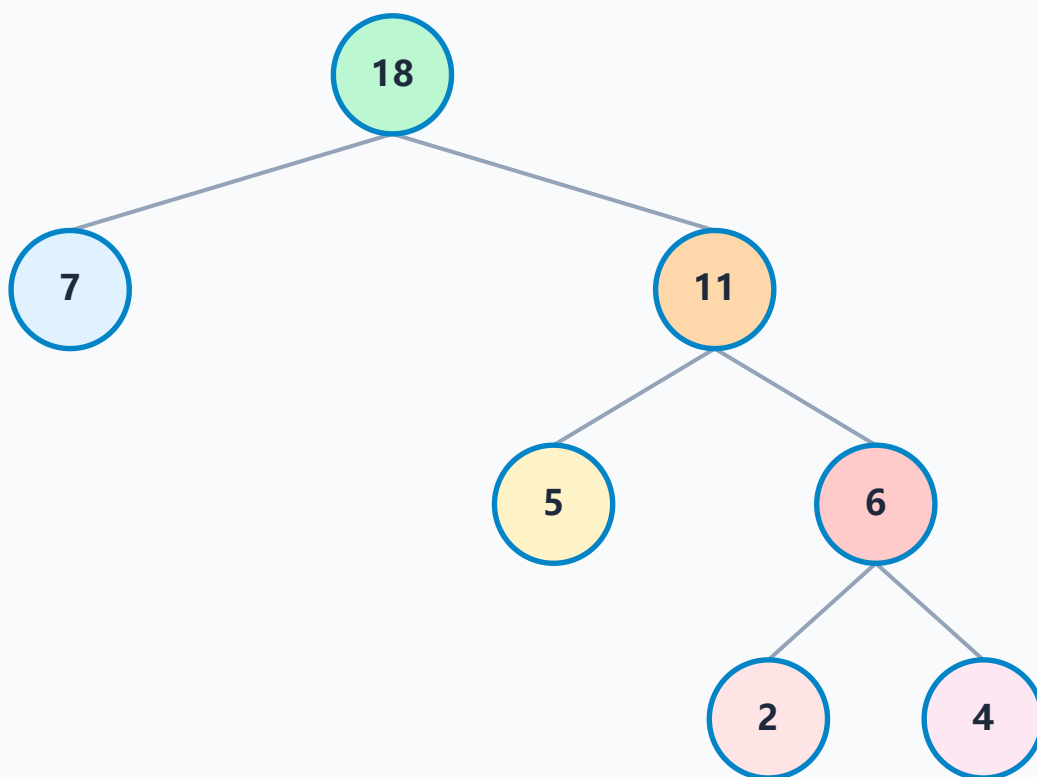
1. 把所有权值看作 n 棵单节点树
2. **每次取权值最小的两棵树合并**，新节点的权值 = 两者之和
3. 重复直到只剩一棵树

💡 每轮都选当前最小的两个 → 这是**贪心算法**思想（真题直接考过）

例子： 权值 {7, 5, 2, 4} 构造哈夫曼树：

① 取 2、4 → 新节点 6 ② 取 5、6 → 新节点 11 ③ 取 7、11 → 根 18

哈夫曼树（权值 {7,5,2,4}）



WPL 计算： 按叶子所在层数算：

叶子	权值	层数 (根为第 0 层)	贡献
7	7	1	$7 \times 1 = 7$
5	5	2	$5 \times 2 = 10$
2	2	3	$2 \times 3 = 6$
4	4	3	$4 \times 3 = 12$

WPL = 7 + 10 + 6 + 12 = 35 

性质（选择题常考）：

- n 个叶子 $\rightarrow$ 哈夫曼树共 $2n - 1$ 个节点
- 哈夫曼树**没有度为 1 的节点**（每个非叶子都有两个孩子）
- 权值越小的叶子**离根越远**（层数越深）

3.3 哈夫曼编码

编码规则： 从根出发，往左子树走记 **0**，往右子树走记 **1**，到叶子停下——路径上的 0/1 串就是该叶子的编码。

对上面 {7, 5, 2, 4} 的树（左 0 右 1）：

叶子	编码
7	0
5	10
2	110
4	111

四大要点（必考）：

1. **前缀编码**：任一字符的编码都不是另一字符编码的前缀 → 解码**无歧义**
(看到 110 一定是 2，不会拆成 1+10)
2. **频率越高，编码不更长（单调性）**：出现次数多的字符，编码长度 $\leq$ 出现次数少的字符
 - 频率相差大 → 编码明显短：如 7 只有 1 位，5 有 2 位，2、4 有 3 位
 - 频率相近 → 编码可能等长：如权值 {6,5,4,3} 中，6 和 5 的编码都是 2 位
3. **排除技巧（选择题利器）**：最高频字符的编码一定是最短的（至少并列最短）——2023 真题里频率 45% 的 f 编码必须是 1 位，据此可直接排除 B、C、D
4. **哈夫曼编码不唯一**：哈夫曼编码只保证 WPL 是最小，但满足该条件的编码方案是很多的，比如左 0 右 1，和左 1 右 0

⚠ **易错点**：左 0 右 1 还是左 1 右 0 是人为约定的，**编码内容会变，但编码长度不变**。题目问“编码长度”时不受左右约定影响。

💡 **进阶结论**：当权值就是字符出现次数时，所有字符的**编码总长度 = WPL**（传输 n 个字符所需的总位数）。

3.4 真题精练

例1（单选） 假设有一组字符 {a,b,c,d,e,f}，对应的频率分别为 5%，9%，12%，13%，16%，45%。以下哪个选项是 a,b,c,d,e,f 分别对应的一组哈夫曼编码？（ ）

- A. 1111, 1110, 101, 100, 110, 0
- B. 1010, 1001, 1000, 011, 010, 00
- C. 000, 001, 010, 011, 10, 11
- D. 1010, 1011, 110, 111, 00, 01

解析： 频率最高的 f (45%) 编码应**最短**——只有 A 给 f 分配了 1 位编码 0，其余选项 f 都是 2 位，直接排除 B、C、D。A 中 a (5%)、b (9%) 频率最低、编码最长 (4 位)，且均为合法前缀编码。选 **A**。

答案：A （真题溯源：2023 CSP-J 初赛第 10 题）

例2（单选） 按哈夫曼算法构造权值 {2, 3, 4, 7} 的哈夫曼树，该树的 WPL 为 ()

- A. 28
- B. 30
- C. 32
- D. 35

解析： 先取 2、3 $\rightarrow$ 5；再取 4、5 $\rightarrow$ 9；最后 7、9 $\rightarrow$ 16。树形：根 16 的左孩子 7（第 1 层），右孩子 9 的左孩子 4（第 2 层）、右孩子 5 的左孩子 2 和右孩子 3（第 3 层）。 $WPL = 7 \times 1 + 4 \times 2 + 2 \times 3 + 3 \times 3 = 7 + 8 + 6 + 9 = 30$ 。选 **B**。

答案：B

例3（单选） 哈夫曼树的构造过程中，每次都选取当前权值最小的两棵树进行合并，这种策略体现的算法思想是 ()

- A. 分治
- B. 回溯
- C. 贪心
- D. 动态规划

解析： 每步都做"当前最优"选择（取两个最小）且不回退，是典型的**贪心**策略。选 **C**。

答案：C

🧠 **本节速记：** 哈夫曼树 = 最优二叉树 (WPL 最小) · 构造：每次取两个最小合并 (贪心) · $WPL = \sum \text{权值} \times \text{层数}$ · n 个叶子 $\rightarrow 2n-1$ 个节点，无一度节点 · 编码：左 0 右 1，前缀编码无歧义，频率高编码不更长 (最高频必最短) · 编码总长度 = WPL

🧠 课堂速记要点

1. **表达式三种写法：** 中缀 $a+b$ 、前缀 $+ab$ 、后缀 $ab+$ ；后缀不需要括号，优先级隐含在顺序中
2. **后缀求值：** 从左到右扫，遇操作数入栈，遇运算符**先弹右操作数 b 、再弹左操作数 a** ，算 $a \text{ op } b$ ，结果入栈
3. **前缀求值：** 从右到左扫，遇运算符**先弹左操作数 a 、再弹右操作数 b** ，算 $a \text{ op } b$
4. **BST 定义：** 左子树 $<$ 根 $<$ 右子树，**递归**成立；判断时要看**整棵子树**，不能只看直接孩子
5. **BST 中序：** 中序遍历 = **严格递增**序列；中序严格递增 $\Leftrightarrow$ 是 BST；第 k 小元素 = 中序第 k 个
6. **哈夫曼树：** WPL 最小的二叉树 (最优二叉树)；构造 = 每次取两个最小合并 (贪心)
7. **哈夫曼性质：** n 个叶子 $\rightarrow 2n-1$ 个节点、无一度节点；编码左 0 右 1、前缀编码无歧义、频率高编码不更长 (最高频必最短)；编码总长度 = WPL



速查答案记忆卡

1	中缀 / 前缀 / 后缀	└→ $a+b$ / $+ab$ / $ab+$
2	后缀求值	└→ 从左到右扫 · 先弹右 b 再弹左 a
3	前缀求值	└→ 从右到左扫 · 先弹左 a 再弹右 b
4	BST 性质	└→ 左 $<$ 根 $<$ 右 (递归成立)
5	BST 中序遍历	└→ 严格递增序列 $\Leftrightarrow$ BST
6	BST 第 k 小元素	└→ 中序遍历第 k 个
7	哈夫曼树	└→ WPL 最小 (最优二叉树)
8	哈夫曼构造	└→ 每次取两个最小合并 (贪心)
9	n 个叶子的节点总数	└→ $2n - 1$
1	哈夫曼编码	└→ 左0右1 · 前缀编码 · 高频编码不更长 (最
0	高频必最短)	
1	编码总长度	└→ $=$ WPL